Lloyd's of London - AI Engineer
Technical Assignment

1. Context

The organisation receives a high volume of legal documents (e.g., notices, lawsuits, formal letters, and other legal correspondence). Today, these documents are processed manually across different teams and regions, resulting in slow throughput, inconsistent outcomes, and limited visibility.

The *Service of Suits* initiative aims to modernise this workflow by developing a **centralised, AI-enabled document-handling capability**, with human oversight, that can scale across jurisdictions.

2. The Assignment

Your goal is to implement a proof of concept that showcases the core mechanics of an AI-assisted document-processing pipeline, demonstrating technical judgement and clear design choices.

The long-term vision is to build an AI-assisted capability that can process legal documents efficiently and transparently, with human oversight. Such a system would ingest documents from multiple sources (email, scanned mail, shared folders, or portals), perform basic format/completeness checks, classify document types, extract key fields (e.g., identifiers, dates, parties), highlight missing or inconsistent information, draft notifications to be sent via email to the relevant recipients, support human validation, and store structured metadata for search, retrieval, and audit.

For this technical exercise, you will implement a thin vertical slice of that vision. Working only with the 6 provided mock PDF documents, build a small end-to-end prototype that:

- loads and processes the PDFs;
- performs simple completeness/format checks;
- extracts structured information (metadata):
    - **Document Type:** Type of legal document (notice, lawsuit, legal correspondence).
    - **Date of Loss:** Date the incident or event occurred.
    - **Policy Number:** Insurance policy identifier referenced in the document.
    - **Recipient:** Party the document is addressed to for action or routing.
    - **Claimant / Defendant:** Main parties bringing the claim and responding to it.

    - **Case / Court Reference Number:** Official external identifier assigned by a court or authority.
- store the metadata listed above in a central repository
- produces a draft email containing the relevant metadata that the legal team can send to the identified recipients
- **Optional:** additional logging for observability and system transparency:
    - request ID
    - Latency
    - Token usage (if applicable)
    - Retrieved chunks (if applicable)
    - Retrieved document titles
    - Any clear logging format or additional information is acceptable.

3. Expected Deliverables

Required

1. **Public GitHub repository (with no secrets).**
2. **Technical Write-Up and additional questions in pdf (2-4 pages) containing the following four sections**

   A. Solution Overview

   - Architecture: One short paragraph and/or a simple diagram of your components. Mention anything you skipped because of the limited time.
   - Flow: 4-6 lines describing the pipeline from ingestion → extraction → output.
   - Any technical choices you made and why (e.g., model choice, retrieval method, guardrails, security, error handling, performance considerations, other).

   B. Assumptions & Trade-offs

   - Any assumptions you made.
   - What you simplified because of the timebox exercise

   C. If You Had More Time (as a short reflection)

   Briefly explain what you would add next (e.g., agentic workflows, additional features, evaluation tests, better ranking, safety checks, caching, adversarial testing, corrective loops). One or two paragraphs is enough.

   D. Enterprise-Grade Architecture

   Describe how this could evolve in a production solution using Databricks + Azure. The solution shall handle ~15.000 documents per year (this might be pdf, Word, emails,

attachments, scanned documents).

One or two paragraph or tiny diagram is fine.

Optional:

- Screenshots with explanations (logs, UI, dashboards, terminal output…).
- Hosted demo (public or password-protected).
- Short demo video (e.g. YouTube unlisted link, Vimeo password protected or similar).

4. Technical Guidelines

Timeboxing

- Aim to allocate about as much time as you would for a morning "hackathon."
- You must be the sole producer of the submission (no collaboration with others)
- If incomplete, describe *what you would do next* and *why*.

Tech Stack

- Any reasonable tech stack, libraries, and AI models are allowed.
- CLI or minimal web UI, your choice.
- You can use AI coding agents/assistants, but must disclose where and how.

Data

- Use only the mock PDFs provided.
- Do not use external data or enhance documents with outside knowledge.

Security & Privacy

- Do not commit secrets to the repo.
- Use environment variables or a Key-Vault-style pattern.

Running the Code

- The reviewer is **not expected to run your code locally**.
- Include clear reproducibility instructions, but design your submission so that it can be **understood, inspected, and evaluated** even without execution.

We are **not** expecting a production-ready system. Instead, your goal is to demonstrate how such a solution *could* operate at scale by implementing a focused, well-reasoned, and observable slice of the overall pipeline. We are evaluating your technical judgement, your ability to design AI-enabled workflows, and your engineering discipline under real-world constraints.

Partial implementations are welcome, provided they are supported by clear design notes explaining how unimplemented components would function. If you run out of time, ensure all assumptions, omissions, and trade-offs are explicitly documented.